

MAI Project Plan

MAI — Personal Jarvis-like AI Assistant

Project Architecture & 30-Day v1 Build Plan

MAI = a conversational AI brain + agent system + tools + memory + voice/UI + extensible add-ons.

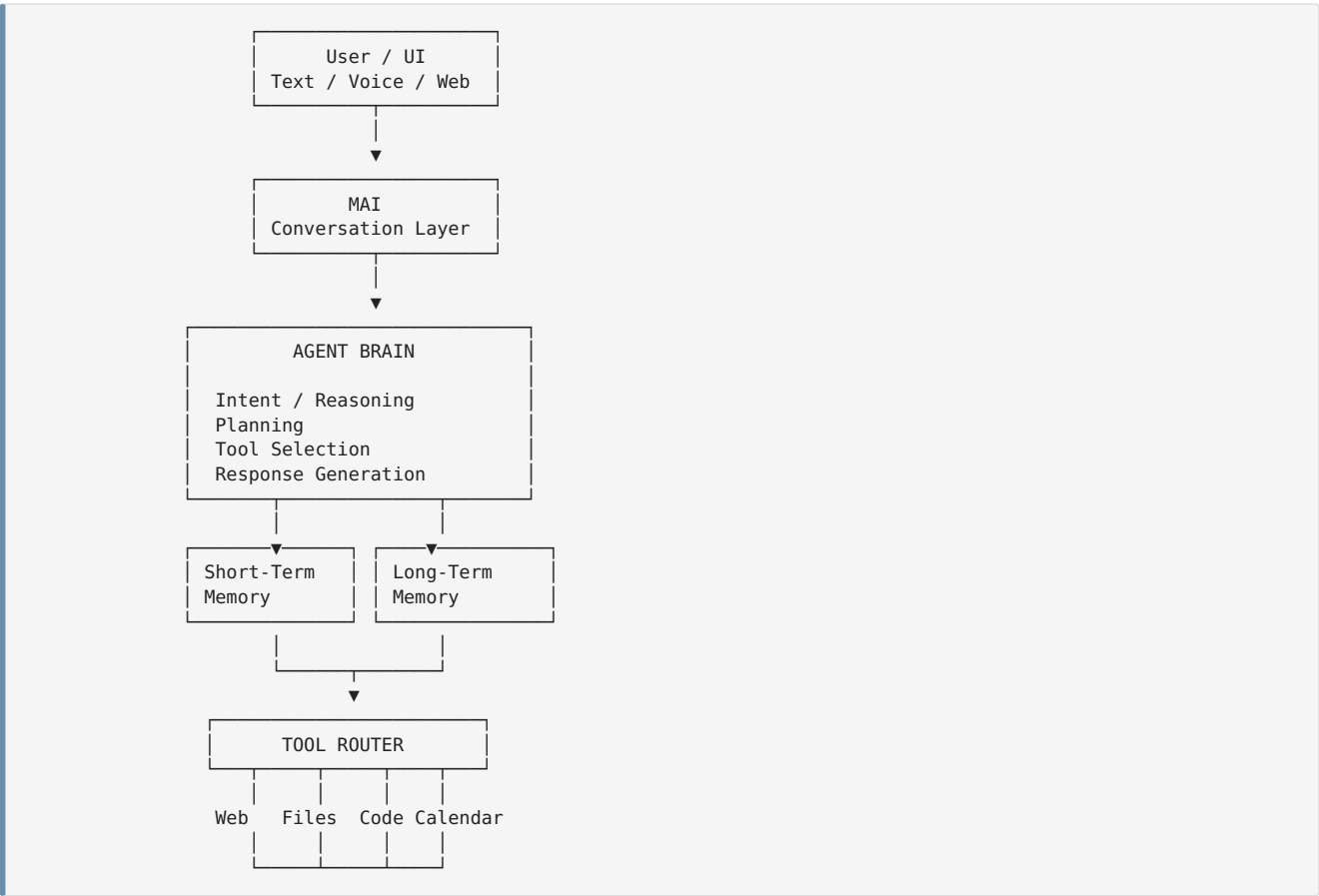
Example of the end goal:

“MAI, check my emails, summarize important ones, look at my calendar, tell me what I should focus on today, and remind me about unfinished work.”

MAI should decide **which tools to use**, execute them, remember relevant things, and respond naturally.

This is a big project. The core recommendation: **don’t start by building 20 tools. Build the brain first, then add capabilities one by one.**

1. The MAI Architecture



The Important Idea

The LLM should not do everything.

Component	Role
LLM	Understands and reasons
Agent framework	Controls the workflow
Tools	Perform actions
Memory system	Remembers useful information
Database	Persists data
UI / Voice	Interacts with the user

That separation makes MAI much easier to grow.

2. Recommended Tech Stack

Designed as a **hybrid system** for a laptop with 16 GB RAM and no dedicated GPU.

Backend — Python + FastAPI

- Excellent AI ecosystem
- Easy API development
- LangGraph / LangChain support
- Good database integrations
- Easy tool development

Agent Orchestration — LangGraph

Needed for:

- Multiple agent/tool steps
- State management
- Conversation persistence
- Retries
- Conditional workflows
- Human approval for sensitive actions

LangGraph supports short-term thread-scoped memory and long-term memory across conversations; its persistence model separates checkpoints from durable stores.

Database

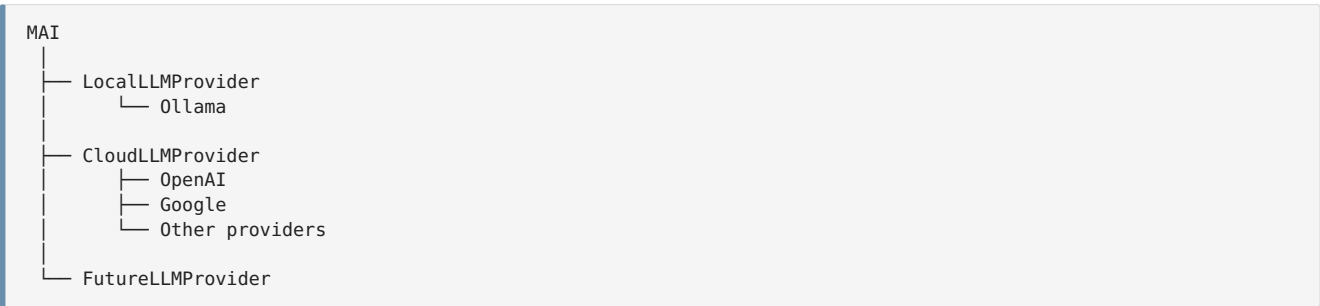
PostgreSQL for: - Users - Conversations - Messages - Tool execution history - Tasks - Settings - Structured memory

pgvector for: - Semantic memory search - RAG - Embeddings - Relevant past conversations

This keeps the early architecture simple — no need for five separate databases.

Local LLM Runtime — Ollama

A practical choice for local development. MAI should not depend on a single model provider — build an abstraction layer instead:



This lets you change the brain without rewriting the application.

3. Which LLM Should You Use?

Don't look for one magical model that does everything — use a model strategy instead.

Option A — Best Starting Point for a Laptop

Try a **small instruction-tuned model in the roughly 4B-8B range**, using quantization appropriate for available RAM. On a 16 GB RAM machine without a GPU, smaller local models are more realistic than large ones.

Local model

↓

Fast/simple conversation
Basic classification
Simple tool decisions

Cloud model

↓

Complex reasoning
Complex planning
Difficult coding
High-quality responses

Option B — Model Roles

Job	Model Type
Friendly conversation	Fast chat/instruct model
Complex reasoning	Strong cloud reasoning model
Tool selection	Small/medium reliable model
Memory extraction	Small cheap model
Embeddings	Dedicated embedding model
Speech-to-text	Dedicated speech model
Text-to-speech	Dedicated TTS model

Do not use the main LLM for everything. Example — extracting the sentence:

"My preferred programming language is C++, and I have an Amazon assessment next week."

...doesn't need the strongest model. A smaller extraction pipeline can produce:

```
{
  "memory_candidates": [
    {
      "type": "preference",
      "content": "User prefers C++",
      "confidence": 0.95
    },
    {
      "type": "event",
      "content": "User has an Amazon assessment next week",
      "confidence": 0.9
    }
  ]
}
```

MAI then decides what should actually be stored.

4. Conversation Memory Architecture

Four levels of memory.

Level 1 — Working Memory

The immediate conversation, scoped to a `thread_id`:

```
User: Help me prepare for an interview
MAI: Sure. Which company?
User: Amazon
MAI: ...
```

LangGraph's short-term memory/checkpointing model is designed for this kind of thread-level conversational state.

Level 2 — Conversation Summary

Don't send a thousand old messages to the LLM.

```
Recent Messages:
- last 10–20 messages

Conversation Summary:
- User is preparing for Amazon SDE roles
- Discussed DSA and aptitude
- Currently focusing on arrays
```

When the conversation grows large:

```
Old messages
  ↓
Summarizer
  ↓
Conversation summary
  ↓
Store
```

Then keep, in context:

```
System Prompt
+
Relevant Long-Term Memory
+
Conversation Summary
+
Recent Messages
```

Level 3 — Long-Term User Memory

The Jarvis-like memory:

```
Preferences
├─ prefers C++
├─ prefers detailed explanations
└─ learning style

Goals
├─ prepare for placements
└─ build MAI

Projects
├─ MAI
└─ other projects

Facts
├─ important stable information
└─ relevant context
```

Don't store everything — this is a common mistake.

- Bad: "User said they ate dosa at 8:30 PM." (probably useless)
- Good: "User is building a personal AI assistant called MAI." (useful across future conversations)

Level 4 — Episodic Memory

Stores important events:

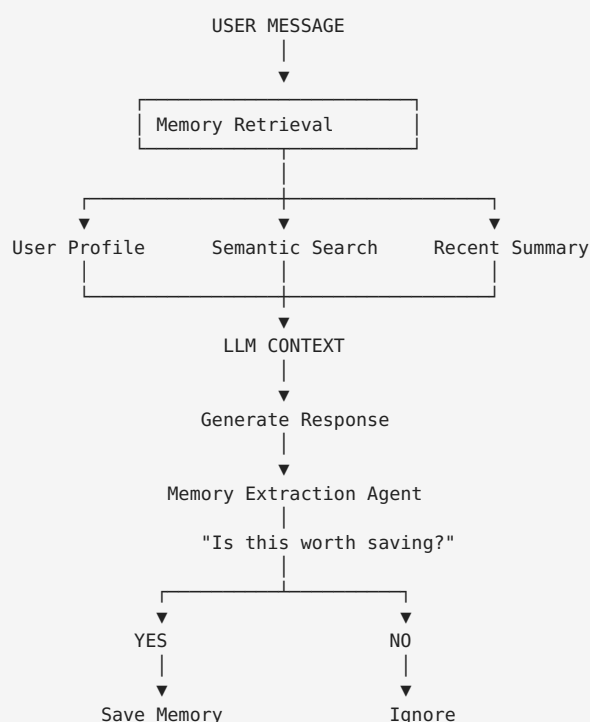
2026-08-29:
User designed MAI architecture.

2026-09-05:
User added web search tool.

2026-09-10:
MAI memory system was implemented.

Later: “What did we do last week on MAI?” — MAI retrieves those episodes. LangGraph’s memory concepts distinguish persistent long-term memory and different kinds of stored knowledge, including semantic, episodic, and procedural-style memories.

5. Recommended Memory Design

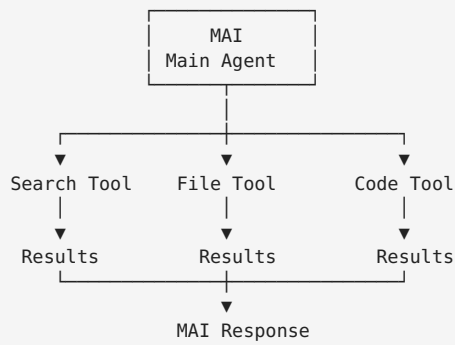


For MAI v1: use **PostgreSQL + pgvector + custom memory logic** before adding another dependency. Later, if a specialized memory layer is wanted, Mem0 provides an agent-focused memory system with long-term memory and retrieval patterns.

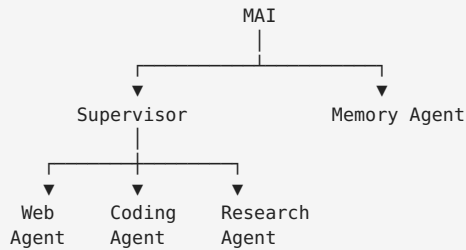
Recommendation: learn the memory architecture first, then evaluate Mem0 — otherwise you’ll use a library without understanding what problem it solves.

6. MAI’s Agent Architecture

Don’t start with “multi-agent.” Start with **one main agent + tools**.



Later, when complexity increases:



But this is **not Day 1**. Multi-agent systems add complexity: more prompts, more latency, more tokens, debugging difficulty, agent loops. First prove that a single agent works.

7. Tool Roadmap — Phase 1 (Core Tools)

1. Web Search Tool

```

User: "What is the latest news about AI?"
MAI
↓
Decides search required
↓
Calls Web Tool
↓
Reads results
↓
Responds
  
```

2. Calculator Tool

Never let the LLM calculate important arithmetic itself.

```
calculate("1298 * 45 / 6")
```

3. Date and Time Tool

```

"What time is it?"
"How many days until October 10?"
  
```

4. File / RAG Tool

MAI can answer questions about PDF, DOCX, TXT, CSV, and personal notes.

```
Upload File
↓
Extract Text
↓
Chunk
↓
Embed
↓
Vector Database
↓
Retrieve Relevant Chunks
↓
LLM Answer
```

5. Memory Tool

Explicit tools the agent can consciously call:

```
remember()
search_memory()
update_memory()
forget_memory()
```

8. Phase 2 Tools

After the core system works, add:

- Email integration
- Calendar integration
- To-do / task manager
- Weather
- News
- GitHub
- File management
- Browser automation

Important: actions like deleting files or sending emails need confirmation.

```
MAI:
"I found the email and prepared the reply.
Do you want me to send it?"
```

Not: "MAI automatically sends it."

Permission Layer

Action Type	Behavior
READ	Automatically allowed
WRITE	Ask confirmation
DELETE	Strong confirmation
FINANCIAL	Always confirm

9. Phase 3 — Jarvis Features

Voice Pipeline

```
Your Voice
↓
Speech-to-Text
↓
MAI Agent
↓
Tools + Memory
↓
Response
↓
Text-to-Speech
↓
MAI Voice
```

Later, add a wake word:

```
Wake Word
↓
"Hey MAI"
↓
Speech Recognition
```

Don't start with a wake word — build text MAI first.

10. Recommended Project Structure

```
mai/
├── backend/
│   ├── main.py
│   ├── agent/
│   │   ├── main_agent.py
│   │   ├── graph.py
│   │   ├── prompts.py
│   │   └── state.py
│   ├── llm/
│   │   ├── base.py
│   │   ├── ollama.py
│   │   └── cloud.py
│   ├── memory/
│   │   ├── short_term.py
│   │   ├── long_term.py
│   │   ├── retrieval.py
│   │   └── extractor.py
│   ├── tools/
│   │   ├── calculator.py
│   │   ├── web.py
│   │   ├── files.py
│   │   ├── datetime.py
│   │   └── memory.py
│   ├── database/
│   │   ├── models.py
│   │   └── connection.py
│   └── api/
│       └── routes.py
├── frontend/
│   └── ...
├── tests/
├── docker-compose.yml
└── README.md
```

Keep this modular from the beginning. Don't put the entire project inside `main.py` — that will hurt badly once tools and memory start increasing.

11. The 30-Day Roadmap

Week 1 — Build the Foundation

Day	Focus	Deliverable / Goal
1	Project design	README, architecture diagram, feature roadmap, DB schema
2	Python backend	FastAPI with <code>GET /health</code> , <code>POST /chat</code>
3	LLM integration	User → API → LLM → Response, one provider abstraction. Goal: MAI can chat
4	Conversation storage	Tables: <code>users</code> , <code>conversations</code> , <code>messages</code> — every message stored
5	Short-term memory	<code>thread_id</code> + history; “My name is Sri” → “What is my name?” answered correctly
6	Conversation summarization	Old messages summarized and stored when they exceed a limit
7	Week 1 testing	Multiple threads, DB persistence, backend restart, conversation resumes

Week 1 result: a working conversational MAI.

Week 2 — Agent and Tools

Day	Focus	Details
8	Learn tool calling	Build <code>calculator_tool()</code> ; teach MAI when to call it
9	Multiple tools	Add calculator, datetime, basic utility; test tool selection
10	Agent loop	User Question → LLM → Need Tool? → (Call Tool → LLM) or Respond → Final Answer
11	Web search	MAI recognizes when a search is required instead of hallucinating
12	Tool registry	<code>TTOOLS = {"calculator": ..., "web_search": ..., "get_time": ...}</code>
13	Error handling	Tool failures, internet failures, invalid arguments, nonexistent tool calls
14	Week 2 testing	30 test prompts across math, latest info, normal chat, and time queries

Week 2 result: MAI can reason about when it needs tools.

Week 3 — Long-Term Memory

Day	Focus	Details
15	Memory schema	<code>memories</code> table: id, user_id, content, type, importance, confidence, created_at, updated_at
16	Memory extraction	"I prefer C++ over Python" → <code>{type: preference, content: "User prefers C++", importance: high}</code>
17	Embeddings	Embeddings, cosine similarity, semantic search; store vectors in pgvector
18	Memory retrieval	Query → Semantic Memory Search → Relevant Memory → Add to Context → LLM
19	Memory filtering	"Should remember?" decision gate: YES → Save, NO → Ignore
20	Memory update/conflict	New preference updates the old one instead of storing both blindly
21	Week 3 testing	Memory across conversations, incorrect memory, changed preferences, forgetting, irrelevant info

Week 3 result: MAI actually remembers useful information.

Week 4 — RAG + Polish

Day	Focus	Details
22	File upload	Support PDF, TXT, DOCX
23	RAG pipeline	Document → Chunk → Embedding → Vector DB → Search → Context → Answer
24	Source-aware answers	MAI states which document it used — important for trust
25	Basic frontend	Chat window, input, conversation list, loading state (not polished yet)
26	Tool activity UI	Show "MAI is thinking...", "Searching the web...", "Reading document..."
27	Permissions	Safe read → auto execute; write/delete → confirm
28	Logging	Store user input, agent decisions, tool calls/results, errors, latency
29	Evaluation	Benchmark: 10 conversation, 10 memory, 10 tool-selection, 10 RAG, 10 error tests
30	MAI v1 release	See checklist below

MAI v1 Release Checklist

- ☐ Chat naturally
- ☐ Remember current conversation
- ☐ Remember selected long-term information
- ☐ Search memories
- ☐ Use tools
- ☐ Search the web
- ☐ Answer from uploaded documents
- ☐ Maintain multiple conversations
- ☐ Persist data
- ☐ Ask permission for sensitive actions

12. Final Recommended Stack for MAI v1

Frontend
└─ React / Next.js
Backend
└─ Python
└─ FastAPI
Agent
└─ LangGraph
Local Model Runtime
└─ Ollama
Database
└─ PostgreSQL
└─ pgvector
Memory
└─ Short-term: LangGraph checkpoints
└─ Conversation summaries
└─ Long-term: PostgreSQL + pgvector
Tools
└─ Calculator
└─ Date/Time
└─ Web Search
└─ Memory
└─ File/RAG
Later
└─ Email
└─ Calendar
└─ GitHub
└─ Voice
└─ Browser automation

LangGraph’s documented model supports exactly this distinction: thread-level short-term persistence plus long-term cross-thread storage, with strategies such as trimming and summarizing history as context grows.

13. The Most Important Rule for MAI — Build in Stages

Stage	Description	Composition
1	Chatbot	User ↔ LLM
2	Conversational assistant	User ↔ LLM + Short-term Memory
3	Personal assistant	User ↔ LLM + Long-term Memory
4	Agent	User ↔ LLM + Memory + Tools
5	Agentic assistant	Understands goal → Plans → Uses multiple tools → Checks results → Continues if necessary → Reports back
6	Jarvis-like MAI	Voice + Memory + Tools + Agents + Personalization

Bottom line: don’t try to build the full Jarvis experience immediately. Build **MAI v1 as an excellent text-based agent with memory and 5 tools**. Once that architecture is solid, voice, add-ons, more agents, and integrations become much easier to layer on.